Building and Learning a Bayesian Network with an LLM — Report

1. Introduction

My main result: none of the code written by the small local LLM was correct, even after I fed its errors back to it. Every correct answer in this report comes from programs I checked against the mathematical specification.

In this assignment I built the classic Sprinkler Bayesian network, did exact inference and learned its parameters from data. I also tested whether a coding LLM can write this code for me. The network has four binary variables (0 = False, 1 = True): Cloudy (C), Rain (R), Sprinkler (S) and WetGrass (W). Its edges are C → R, C → S, R → W and S → W, so the joint distribution factorises as:

```
P(C,R,S,W) = P(C)\,P(R\mid C)\,P(S\mid C)\,P(W\mid R,S)
```

The parameters I was given:

Quantity	Value
P(C=1)	0.5
P(R=1 given C=0) / P(R=1 given C=1)	0.2 / 0.8
P(S=1 given C=0) / P(S=1 given C=1)	0.5 / 0.1
P(W=1 given R,S) for (0,0), (0,1), (1,0), (1,1)	0.01, 0.90, 0.90, 0.99

The course separates two jobs. **AI science** is defining the probability model and doing correct inference. **AI engineering** is using an LLM to turn a specification into code, then checking and fixing that code. The pipeline I followed was: specify, generate, inspect, execute, validate, test. The key rule is that the LLM only writes code; `pgmpy` does the actual inference, and my tests decide whether the code is accepted.

2. Setup and environment

I ran everything on my own laptop, which has no GPU, so the LLM was slow: about 2 tokens per second, or 1 to 6 minutes per answer.

```
Python      : 3.13.14 Windows
pgmpy       : 1.1.2
transformers: 5.17.0
torch       : 2.14.0+cpu | CUDA available: False
```

- **LLM:** `Qwen/Qwen2.5-Coder-1.5B-Instruct` (about 3 GB), loaded with the Hugging Face `transformers` pipeline. I used greedy decoding (`do_sample=False`) so the output is reproducible.
- **Caching:** because each answer took minutes, I added a small wrapper around `ask_coder` that saves every prompt and response to `llm_responses.json`. Re-running the notebook re-uses those answers, so it reproduces exactly the same outputs in about 2 minutes. The file is also a record of what the LLM really said.
- **Library versions matter:** pgmpy 1.x removed the old `BayesianModel` class, and it now wants estimator objects such as `DiscreteMLE()`. I printed the versions at the top of the notebook because generated code has to be tested against the installed API.

3. The trusted reference model and exact inference

My own implementation gives P(Rain=1 | WetGrass=1) = 0.7048, and brute-force enumeration agrees to within 2.2×10^{-16}.

Before using any LLM, I built the network myself with `DiscreteBayesianNetwork` and `TabularCPD`, so later I had something to compare against. In a `TabularCPD`, each **row** is a state of the variable, and each **column** is one combination of parent states. With `evidence=["Rain", "Sprinkler"]`, the columns go (R=0,S=0), (R=0,S=1), (R=1,S=0), (R=1,S=1), because the last parent changes fastest. `check_model()` returned True.

```
+-------------+--------------+--------------+--------------+--------------+
| Rain        | Rain(0)      | Rain(0)      | Rain(1)      | Rain(1)      |
| Sprinkler   | Sprinkler(0) | Sprinkler(1) | Sprinkler(0) | Sprinkler(1) |
+-------------+--------------+--------------+--------------+--------------+
| WetGrass(0) | 0.99         | 0.1          | 0.1          | 0.01         |
| WetGrass(1) | 0.01         | 0.9          | 0.9          | 0.99         |
+-------------+--------------+--------------+--------------+--------------+
```

For inference I used **variable elimination**. It fixes the evidence W=1, multiplies the relevant factors, sums out the hidden variables C and S, and then normalises:

```
P(R \mid W=1) = \frac{\sum_C \sum_S P(C)\,P(R\mid C)\,P(S\mid C)\,P(W=1\mid R,S)}{\sum_R \sum_C \sum_S P(C)\,P(R\mid C)\,P(S\mid C)\,P(W=1\mid R,S)}
```

```
+---------+-------------+
| Rain    |   phi(Rain) |
+=========+=============+
| Rain(0) |      0.2952 |
| Rain(1) |      0.7048 |
+---------+-------------+
P(Rain=1 | WetGrass=1) = 0.7047692307692307
```

A program that runs is not automatically correct, so I checked this independently. I enumerated all $2^4 = 16$ assignments with plain Python functions for each factor:

```
Enumeration: 0.7047692307692309
pgmpy      : 0.7047692307692307
Difference : 2.220446049250313e-16
```

The difference is just floating-point rounding, so the library result is confirmed. This enumeration became my **test oracle** for the rest of the assignment. Rain goes up from its prior of 0.5 to 0.70 because wet grass is evidence for rain.

4. Asking the LLM to write the inference code

The LLM understood the question (the query is Rain, the evidence is WetGrass=1) but got the network wrong and used an outdated API. My corrected program matches the reference exactly.

I gave the LLM a very explicit prompt: the edges, all nine probabilities, and requirements such as "use `DiscreteBayesianNetwork` and `VariableElimination`" and "store the result in `generated_posterior`". Here is part of what it returned:

```
from pgmpy.models import BayesianModel
generated_model = BayesianModel()
...
rain_cpd = TabularCPD('Rain', 2, [[0.2, 0.8], [0.8, 0.2]], evidence=
['Cloudy'], evidence_card=[2])
sprinkler_cpd = TabularCPD('Sprinkler', 2, [[0.5, 0.1], [0.1, 0.5]],
evidence=['Cloudy'], evidence_card=[2])
wet_grass_cpd = TabularCPD('WetGrass', 2, [
    [0.01, 0.90, 0.90, 0.01],
    [0.90, 0.01, 0.90, 0.90]
], evidence=['Rain', 'Sprinkler'], evidence_card=[2])
...
query_result = generated_model.query(variables=['Rain'], evidence=
{'WetGrass': 1})
generated_posterior = query_result['Rain']
```

What I found when I inspected it

Problem in the LLM code	Why it is wrong
`BayesianModel`	removed in pgmpy 1.0, so it gives an ImportError
Rain rows `[[0.2, 0.8], [0.8, 0.2]]`	the rows are swapped: it says P(R=1 given C=0) = 0.8, which is the opposite of the spec
Sprinkler columns	each column sums to 0.6, so it is not a probability distribution
WetGrass values	the P(W=1) numbers are in the W=0 row, one entry is 0.01 instead of 0.99, and the columns do not sum to 1
`evidence_card=[2]` for two parents	should be `[2, 2]`
`generated_model.query(...)`	models have no `query` method; `VariableElimination` was never used

The notebook's basic static check (it only looks for bare calls such as `eval` and `open`) reported **no problems**. That is because it checks safety, not correctness.

Running it and checking it

After inspecting the code, I set `APPROVE_GENERATED_CODE = True`, but I wrapped the execution so that a crash would be recorded instead of stopping the notebook.

```
Generated code FAILED at run time -> ImportError: cannot import name
'BayesianModel' from 'pgmpy.models'
VALIDATION FAILED: AssertionError('generated_model was not created.')
```

I also wrote a stronger validator. It checks the nodes and edges, calls `check_model()`, and compares **every CPT entry by name** with the spec (using `get_value`, so parent order does not matter). It then checks the posterior against the enumeration oracle. To make sure the validator itself works, I first ran it on my trusted model, which passed.

Letting the LLM repair its code

Next, I added a repair loop. If the code fails, the LLM gets back the original prompt, its own code, the exact error message and a few facts about the installed pgmpy API, and tries again (at most 2 rounds). The validator, not the LLM, decides what passes.

```
--- round 0: FAIL  ImportError: cannot import name 'BayesianModel' from
'pgmpy.models'
--- round 1: FAIL  ValueError: Length of evidence_card doesn't match length
of evidence
--- round 2: FAIL  ValueError: Length of evidence_card doesn't match length
of evidence
```

The feedback fixed the API: the LLM switched to `DiscreteBayesianNetwork` and `VariableElimination`. But it repeated the same `evidence_card` bug even though the error message named it directly. To see what was hiding behind that crash, I fixed **only** that one line myself:

```
execution  -> ValueError: Sum or integral of conditional probabilities for
node Sprinkler is not equal to 1.
CPT check  -> CPT entries differ from the specification: P(Rain=1 |
{'Cloudy': 0}) = 0.8, spec says 0.2;
              P(Rain=1 | {'Cloudy': 1}) = 0.2, spec says 0.8; P(Sprinkler=1 |
{'Cloudy': 0}) = 0.1, spec says 0.5; ...
              P(WetGrass=1 | {'Rain': 0, 'Sprinkler': 0}) = 0.9, spec says
0.01; ...
```

So even if the program had run, it would have represented a completely different distribution.

My corrected version

I wrote a corrected program with the same variable names, and it went through exactly the same checks:

```
execution error: None
{'structure': 'PASS', 'CPTs': 'PASS', 'check_model': 'PASS', 'posterior':
0.7047692307692307,
 'reference': 0.7047692307692309, 'absolute_error': 2.220446049250313e-16}
```

5. Learning the parameters from data (maximum likelihood)

With 1,000 simulated samples, MLE estimated P(Rain=1 | Cloudy=1) as 0.7725 (true value 0.8), and my manual count gives exactly the same number.

In this part the graph is known but the CPT numbers are not. For complete data, the maximum-likelihood estimate is just a relative frequency, for example:

```
\hat{P}(R=1 \mid C=1) = \frac{N(R=1,\,C=1)}{N(C=1)}
```

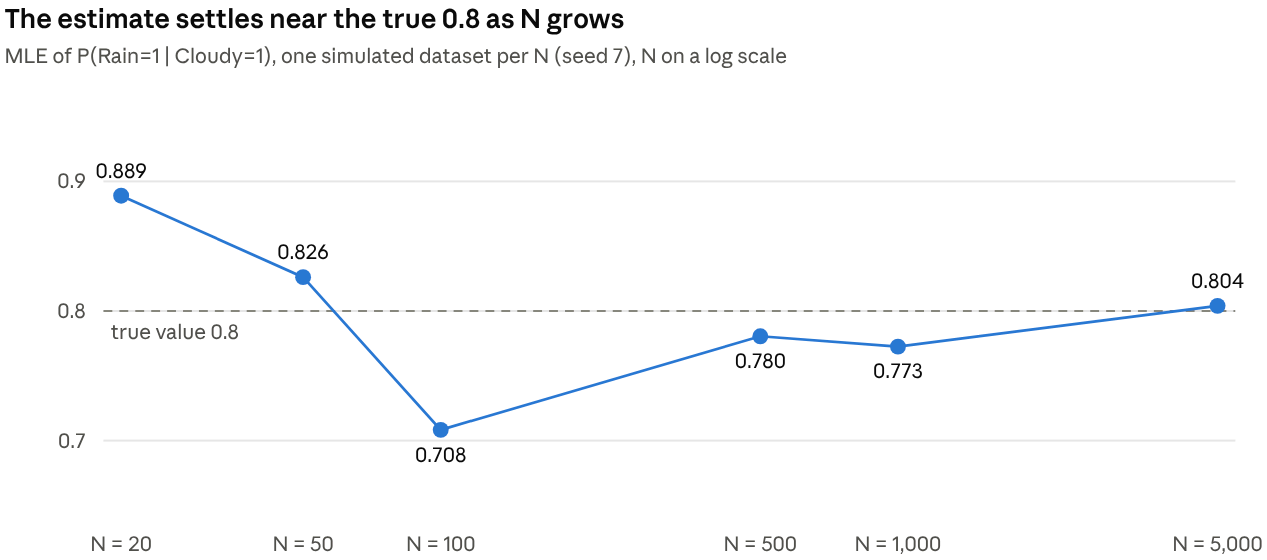
I simulated 1,000 samples from the reference model (seed 7) and fitted the empty structure with `DiscreteMLE()`:

```
Valid fitted model: True
+-----------+-------+
| Cloudy(0) | 0.512 |
| Cloudy(1) | 0.488 |
+-----------+-------+
+---------+-------------+---------------------+
| Cloudy  | Cloudy(0)   | Cloudy(1)           |
+---------+-------------+---------------------+
| Rain(0) | 0.810546875 | 0.22745901639344263 |
| Rain(1) | 0.189453125 | 0.7725409836065574  |
+---------+-------------+---------------------+
```

Then I checked one entry by counting rows in the DataFrame myself:

```
Manual estimate: 0.7725409836065574
pgmpy estimate : 0.7725409836065574
True value     : 0.8
```

Effect of sample size



notebook output · fit_mle_from_n, N = 20 to 5,000, seed 7

With small samples the estimate jumps around: it is 0.889 at N = 20 and 0.708 at N = 100. By N = 5,000 it is within 0.004 of the truth. This is because each estimate is an average over a finite random sample.

The LLM's estimation code

I asked the LLM to fit the CPTs with `MaximumLikelihoodEstimator`, keeping the graph fixed and using the existing `data` DataFrame. What it returned did not actually estimate anything:

- it used the removed `BayesianModel` class;
- even though the prompt said the CPTs are unknown, it **hard-coded made-up CPTs**, for example `Rain` with `[[0.4, 0.6], [0.6, 0.4]]`, and it left out the parents of Rain, Sprinkler and WetGrass;
- it called `mle.estimate()`, which does not exist (the method is `get_parameters()`).

```
Generated estimation code NOT accepted -> ImportError: cannot import name
'BayesianModel' from 'pgmpy.models'
--- round 0: FAIL  ImportError: cannot import name 'BayesianModel' ...
--- round 1: FAIL  AttributeError: 'list' object has no attribute 'shape'
--- round 2: FAIL  AttributeError: 'list' object has no attribute 'shape'
```

The estimation validator checks that the graph is unchanged, that `data` was not modified or replaced, and that every fitted CPT equals the trusted fit on the same data. My corrected version builds the fixed graph and adds the CPDs from `MaximumLikelihoodEstimator(model, data).get_parameters()`. It passes:

```
{'structure': 'PASS', 'data untouched': 'PASS', 'max CPT difference': 0.0}
    variable  max absolute difference
0     Cloudy                      0.0
1       Rain                      0.0
2  Sprinkler                      0.0
3   WetGrass                      0.0
```

6. Sparse data and Bayesian (BDeu) estimation

With only 30 samples, MLE gave P(Rain=1 | Cloudy=1) = 0.909, but BDeu with an equivalent sample size of 10 gave 0.781, much closer to the true 0.8.

With small datasets, some parent combinations are rare, so MLE can give extreme values. A Bayesian estimator adds prior pseudo-counts to the observed counts (posterior counts = observed counts + prior pseudo-counts). This pulls the estimates towards uniform.

```
MLE Rain CPD                                    Bayesian (BDeu, ESS=10) Rain
CPD
| Cloudy  | Cloudy(0) | Cloudy(1) |            | Cloudy  | Cloudy(0) |
Cloudy(1) |
| Rain(0) | 0.947     | 0.091     |            | Rain(0) | 0.854     | 0.219
|
| Rain(1) | 0.053     | 0.909     |            | Rain(1) | 0.146     | 0.781
|
```

Method	P(Rain=1 given Cloudy=1)
True parameter	0.800

Method	P(Rain=1 given Cloudy=1)
MLE, N = 30	0.909
BDeu, N = 30	0.781

This does not mean BDeu is always better. The prior is an extra modelling assumption, so it has to be stated openly rather than hidden inside the code.

The LLM's BDeu code was unsafe

The LLM's program did this:

```
data = pd.read_csv('your_data.csv')  # Replace 'your_data.csv' with your
actual file path
estimator = BayesianEstimator(data, ['Cloudy', 'Rain', 'Sprinkler',
'WetGrass'])
generated_bayes_model = estimator.fit(prior_type="BDeu",
equivalent_sample_size=10)
```

It **reads a file** (which the prompt forbids), **throws away the `data` we provided**, never builds the graph, and calls the estimator with the wrong arguments. The notebook's basic check missed all of this because `pd.read_csv` is a method call, not a bare `open(...)`. So I wrote a stricter check that also flags `read_*` / `to_*` method calls and reassigning `data`:

```
Basic check   : no listed problems
Stricter check: ["Overwrites the provided variable 'data'", 'Suspicious
attribute call (file/system access?): .read_csv(...)']
```

In all three repair rounds the LLM kept the `read_csv` line. In the end it gave up on estimation completely and hard-coded made-up CPTs, so every round was rejected.

Does my validator actually check the prior?

A test that accepts anything is useless. So I fitted models with three different settings and checked that only the requested prior passes:

```
MLE           -> rejected
BDeu, ESS=5   -> rejected
BDeu, ESS=10  -> accepted
```

My corrected program builds the fixed graph, then uses `BayesianEstimator(model, data).get_parameters(prior_type="BDeu", equivalent_sample_size=10)`. It passes with a maximum CPT difference of 0.0 from the trusted fit.

7. Why validation needs semantic tests

`check_model()` catches numbers that do not add up, but it accepted a model with the WetGrass columns in the wrong order. That model gives P(Rain=1 | WetGrass=1) = 0.7173 instead of 0.7048, and only my CPT-level check caught it.

Case 1: numbers that do not add up. I made a Rain CPT where the Cloudy=0 column sums to 0.9 + 0.3 = 1.2. The library rejected it:

```
Validation caught the problem:
ValueError: Sum or integral of conditional probabilities for node Rain is not
equal to 1.
```

Case 2: valid numbers with the wrong meaning. I swapped the last two WetGrass columns. Every column still sums to 1, so pgmpy was happy:

```
check_model(): True
Correct posterior: 0.7047692307692307
Wrong posterior  : 0.7172952268709487
```

My validator compares each entry by named parent states, so it found exactly which entries were wrong:

```
Semantic validator caught it:
  CPT entries differ from the specification: P(WetGrass=1 | {'Rain': 1,
'Sprinkler': 0}) = 0.99, spec says 0.9;
  P(WetGrass=0 | {'Rain': 1, 'Sprinkler': 0}) = 0.01, spec says 0.1;
P(WetGrass=1 | {'Rain': 1, 'Sprinkler': 1}) = 0.9, spec says 0.99; ...
```

The lesson: a program that runs is not necessarily the correct probabilistic model, and a correct model still needs appropriate assumptions.

Reusable tests. I turned these checks into test functions so they can be re-run at any time:

```
PASS test_reference_model
PASS test_wrong_semantics_is_detected
PASS test_human_revised_programs
PASS test_static_check_rejects_file_access
```

8. HW exercise: changing the query

All three of my predictions were right: P(S=1 | W=1) = 0.4278, P(C=1 | W=1) = 0.5746 and P(R=1 | W=1, S=0) = 0.9922. The LLM translated every query correctly, but none of its programs produced an answer.

Step 1: predictions made before running anything

First I worked out the priors from the spec: P(S=1) = 0.5 × 0.5 + 0.5 × 0.1 = 0.30, P(C=1) = 0.5, and from Section 3, P(R=1 | W=1) = 0.705.

Query	Query variable	Evidence	Compared with	My prediction	My reasoning
P(S=1 given W=1)	Sprinkler	WetGrass=1	P(S=1) = 0.30	increase	Sprinkler causes wet grass, so seeing wet grass makes each of its causes more likely. Rain is the more likely cause, so it should stay below 0.5.
P(C=1 given W=1)	Cloudy	WetGrass=1	P(C=1) = 0.50	small increase	Clouds make rain more likely (wetter) but the sprinkler less likely (drier). P(W=1 given C=1) = 0.747 is bigger than P(W=1 given C=0) = 0.553, so the rain path wins, but the two effects partly cancel.
P(R=1 given W=1, S=0)	Rain	WetGrass=1, Sprinkler=0	P(R=1 given W=1) = 0.705	strong increase, close to 1	Explaining away: the sprinkler was off, so rain is the only reasonable explanation, and P(W=1 given R=0, S=0) is only 0.01.

Step 2: the LLM's code

For each query I sent the full network plus the query and evidence, and asked for the result in `generated_query_result`. When I inspected the code:

- **the query variable and evidence were correct in all three programs**, for example `query(variables=['Rain'], evidence={'WetGrass': 1, 'Sprinkler': 0})`;
- **the network was wrong every time**: the same swapped Rain rows, the same non-normalised Sprinkler and WetGrass CPDs, `BayesianModel`, and `model.query`. For the Cloudy query it even used `variable_card=[1]`;
- after one repair round the API was fixed, but the CPD definitions still crashed (`evidence_card` length, or 1-D `values`).

```
                              query  first attempt  rounds used  final
status  LLM posterior   trusted
          P(Sprinkler=1 | WetGrass=1)           FAIL            1
FAIL            NaN  0.427846
             P(Cloudy=1 | WetGrass=1)           FAIL            1
FAIL            NaN  0.574615
  P(Rain=1 | WetGrass=1, Sprinkler=0)           FAIL            1
FAIL            NaN  0.992202
```

Step 3: trusted answers, checked two ways

I computed every query with `VariableElimination` on the reference model and again by brute-force enumeration. My corrected programs, which only change the query line, passed the same validator.

Query	VariableElimination	Enumeration	Before the evidence	Change	Prediction correct?
P(Sprinkler=1 given WetGrass=1)	0.4278	0.4278	0.3000	+0.1278	yes, increase
P(Cloudy=1 given WetGrass=1)	0.5746	0.5746	0.5000	+0.0746	yes, small increase
P(Rain=1 given WetGrass=1, Sprinkler=0)	0.9922	0.9922	0.7048	+0.2874	yes, strong increase

The two methods agree to within 2.2×10^{-16} for every query.

9. Classroom exercises

The LLM's explanation of the WetGrass CPT was wrong: it swapped the middle two columns and got 3 of the 4 probabilities wrong. Its explanation of dataset variability was mostly right.

9.1 Asking the LLM to explain the WetGrass CPT

I gave the LLM the WetGrass `TabularCPD` code and asked what each column means. Part of its answer:

```
The first column ... given no rain or sprinkler. The second column ... given
rain but no sprinkler.
The third column ... given sprinkler but no rain. The fourth column ... given
both rain and sprinkler.
- P(WetGrass=1 | Rain, Sprinkler) = 0.10
- P(WetGrass=1 | Rain, NoSprinkler) = 0.01
- P(WetGrass=1 | NoRain, Sprinkler) = 0.90
- P(WetGrass=1 | NoRain, NoSprinkler) = 0.99
```

I did not trust this, so I asked pgmpy itself. The CPD's axes are `['WetGrass', 'Rain', 'Sprinkler']` with shape (2, 2, 2), so the last variable (Sprinkler) changes fastest. I then compared the LLM's claims column by column:

Column	True (Rain, Sprinkler)	LLM says	Meaning correct?	True P(W=1)	LLM's P(W=1)	Probability correct?
0	(0, 0)	(0, 0)	yes	0.01	0.99	no
1	(0, 1)	(1, 0)	no	0.90	0.90	yes
2	(1, 0)	(0, 1)	no	0.90	0.01	no
3	(1, 1)	(1, 1)	yes	0.99	0.10	no

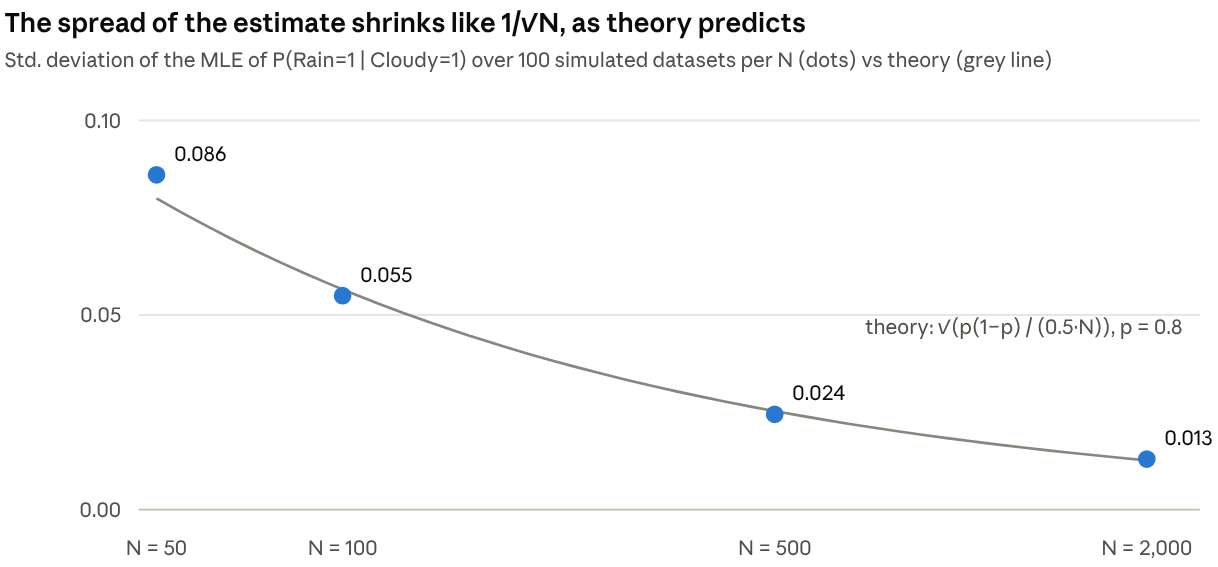
The explanation sounds confident but gets the meaning of the table backwards. It even says grass is almost never wet when both rain and the sprinkler are on. This shows that code explanation is a separate task from code generation, and it can also be wrong.

9.2 Estimating from different datasets

I simulated 5 datasets of 100 samples each (seeds 1 to 5) and fitted MLE on each one:

```
   seed  N(Cloudy=1)  estimate  true value  absolute error  approx. std.
error
0     1           49  0.693878         0.8        0.106122
0.057143
1     2           44  0.795455         0.8        0.004545
0.060302
2     3           47  0.744681         0.8        0.055319
0.058346
3     4           56  0.767857         0.8        0.032143
0.053452
4     5           49  0.795918         0.8        0.004082
0.057143
```

The model, the code and the estimator are identical every time; only the random sample changes. The estimate is a sample proportion computed from only about 50 cloudy rows, so its standard error is about the square root of 0.8 × 0.2 / 50, which is 0.057. That matches how much the values jump. To check this properly, I repeated the experiment with 100 datasets for each of four sample sizes:



notebook output · 100 datasets per N (seeds 1000–1099), N = 50 to 2,000

The mean estimate was close to 0.8 at every N (0.777, 0.806, 0.802, 0.801), so the estimator is essentially unbiased. Its spread falls from 0.086 to 0.013 as N goes from 50 to 2,000, closely following the theoretical curve.

The LLM's explanation. It said the differences come from "inherent randomness" and that variability decreases with more data. Both are correct, and it did not blame pgmpy. However, it also said that the seed changes "the initial conditions for the Bayesian network". That is not right: the network is the same every time, and only the sampled data are different. The correct explanation is sampling variability.

10. Evaluation with the rubric, and what I learned

All 5 groups of LLM-written programs failed my tests, while all of my corrected programs passed.

Program	Structure	Parameters	Inference / estimation	Safety	Verdict
LLM inference code (first try)	right edges, removed `BayesianModel` class	Rain swapped; Sprinkler and WetGrass not normalised; wrong `evidence_card`	right query and evidence, but `model.query` does not exist	passed static checks	FAIL (ImportError)
LLM inference code after 2 repairs	correct	same CPT errors	`VariableElimination` now correct	passed	FAIL (crash; wrong CPTs underneath)
LLM MLE code (+2 repairs)	parents dropped from the CPDs	made-up CPTs	`.estimate()` does not exist	passed	FAIL
LLM BDeu code (+2 repairs)	no graph at first	made-up CPTs	prior named correctly, wrong constructor	**read a CSV and overwrote `data`**	FAIL
LLM HW query programs (3)	right edges	same CPT errors	**query and evidence correct in all 3**	passed	FAIL
LLM explanation of the WetGrass CPT		columns swapped, 3 of 4 values wrong			wrong
LLM explanation of variability			main idea right, one wrong sentence		mostly right

Program	Structure	Parameters	Inference / estimation	Safety	Verdict
My corrected programs (inference, MLE, BDeu, 3 queries)	PASS	PASS (each entry checked)	PASS (error about 2×10^{-16}, CPT difference 0.0)	PASS	PASS

What I learned

1. **If the code runs, that does not mean it is correct.** `check_model()` accepted a model with the wrong meaning. Only semantic tests caught it: comparing each CPT entry, the enumeration oracle, and comparing against a trusted fit.
2. **The small LLM got the shape of the program right, but not the probability content.** It handled the imports, the graph and the query and evidence well. It failed on how CPTs are laid out and on the current pgmpy API, and error feedback fixed the API but not the numbers.
3. **Safety checks have to be stronger than name matching.** The LLM read a CSV file through `pd.read_csv` and replaced my data. The basic check missed this; my stricter check caught it.
4. **The person stays responsible.** I wrote the specification, chose the prior and tested it (BDeu with ESS = 10, checked against other priors), and only accepted code after it passed the tests. The LLM can speed up writing code, but it cannot be the source of probabilistic truth.

Overall, the reliable workflow is specify → generate → inspect → execute → validate → test, not prompt → trust the answer.

Supporting files: `llm_bn_completed.ipynb` *(the full executed notebook) and* `llm_responses.json` *(every LLM prompt and response).*